Тема 12: Процедурно програмиране – указатели, масиви и рекурсия.

ТЕОРИЯ

1. Указатели

Указател = променлива, съдържаща **адреса** на друга променлива.

```
<тип>* <ime> [= <стойност>];

int a = 10;
int* ptr = &a;    // ptr сочи към a
int* null_ptr = nullptr;  // неутрална стойност
```

Указателят знае две неща: **адреса** + **типа** на обекта.

Оператор	**Действие**	**Пример**
`&x`	Взима адреса на x	`ptr = &a`
`*ptr`	Дереферира — взима стойността	`*ptr = 20` (а става 20)

```
int a = 10;
int* ptr = &a;
*ptr += 1;
cout << a;   // 11
```

Указателна аритметика

За `T* ptr`:

```
ptr + n      // нов указател: адрес на ptr + n * sizeof(T)
ptr++        // отмества с sizeof(T) напред
ptr--        // отмества назад
ptr[i]   ⟺  *(ptr + i)   // индексиране
```

Сравнение на указатели: ==, !=, <, >, <=, >= (сравняват адреси)

void указател

```
void* ptr;  // може да сочи към всякакъв тип
// Без дереферииране и без аритметика!
```

Константни указатели

```
const int* ptr = &a;   // указател към константа: ptr може да се мени, *ptr – не
int* const ptr = &a;   // константен указател: ptr не може да се мени, *ptr – може
const int* const ptr;  // и двете са константни
```

---

2. Масиви

Масив = наредена последователност от елементи от **един и същ тип** в **непрекъсната памет**.

```
<тип> <ime>[размер] [= {инициализатор}];

int arr[5];             // 5 елемента – случайни стойности!
int arr[5] = {0};       // 5 нули
int arr[] = {1,2,3,4};  // размерът се извежда = 4
```

Размерът трябва да е константа по време на компилация (за статичен масив)!

Индексиране: `arr[i]` = `*(arr + i)`, индекси: `0` до `размер-1`

> Когато пишем само `arr` → **автоматично се преобразува в указател към първия елемент** (`arr ≡ &arr[0]`)

Масивите се подават по адрес — промените в функция се отразяват на оригинала!

```
void fill(int* arr, int n, int val) {
    for (int i = 0; i < n; i++) arr[i] = val;
}
```

Копиране и сравнение — само поелементно (не `arr1 = arr2`):

```
for (int i = 0; i < n; i++) arr2[i] = arr1[i];
```

Динамичен масив (размерът се знае само по runtime)

```
int n;
cin >> n;
int* arr = new int[n];
// ... употреба ...
delete[] arr;   // задължително!
```

Многомерни масиви

```
int matrix[3][4];   // 3 реда × 4 колони = 12 int-а в паметта (ред по ред)

// Инициализация
int m[2][3] = {{1,2,3},{4,5,6}};

// Обхождане
for (int i = 0; i < ROWS; i++)
    for (int j = 0; j < COLS; j++)
        cout << m[i][j] << " ";
```

В паметта: `int m[R][C]` ≡ `int arr[R*C]` — линейна наредба ред по ред.
Достъп: `m[i][j]` = `*(*(m+i)+j)` = елементът на позиция `i*C + j`

Динамична матрица:

```
int** matrix = new int*[rows];
for (int i = 0; i < rows; i++)
    matrix[i] = new int[cols];
// ... употреба ...
for (int i = 0; i < rows; i++) delete[] matrix[i];
delete[] matrix;
```

---

3. Символни низове (C-strings)

```
char str[10] = "abc";
// Памет: ['a']['b']['c']['\0']['\0']...
```

Задължително завършват с `\0` (нула терминатор)!

Когато пишем `str` → указател към първия символ. Итерация:

```
while (*str) {    // докато не стигнем '\0'
    cout << *str;
    str++;
}
```

Основни функции (`<cstring>`):

Функция	**Действие**	**Сложност**
`strlen(s)`	Дължина без `\0`	O(n)
`strcpy(dst, src)`	Копира src в dst (включва `\0`)	O(n)
`strcat(dst, src)`	Конкатенира src накрая на dst	O(n)
`strcmp(a, b)`	0=равни, <0 ако a<b лексикогр., >0 ако a>b	O(n)
`strncpy(dst, src, n)`	Копира макс. n символа	O(n)

Динамичен низ:

```
char* str = new char[strlen(src) + 1];  // +1 за '\0'
strcpy(str, src);
// ...
delete[] str;
```

---

4. Алгоритми за търсене

Линейно търсене — O(n)

```
int linearSearch(int* arr, int n, int target) {
    for (int i = 0; i < n; i++)
        if (arr[i] == target) return i;  // връща индекса
    return -1;  // не е намерен
}
```

Двоично търсене — O(log n) ⚠ Масивът трябва да е сортиран!

```
int binarySearch(int* arr, int n, int target) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;  // избягваме overflow
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) lo = mid + 1;
        else                   hi = mid - 1;
    }
    return -1;
}
```

5. Алгоритми за сортиране

Помощна функция swap:

```
void swap(int& a, int& b) {
    int tmp = a; a = b; b = tmp;
}
```

Метод на мехурчето (Bubble Sort) — O(n²)

Идея: на всяка стъпка "изкарваме" най-големия несортиран елемент до края.

```
void bubbleSort(int* arr, int n) {
    for (int i = 0; i < n; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                swapped = true;
            }
        }
        if (!swapped) return;  // вече сортиран
    }
}
```

Пряка селекция (Selection Sort) — O(n²)

Идея: намираме минималния в несортираната część и го поставяме на правилното място. По-малко размествания от bubble sort.

```
void selectionSort(int* arr, int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx]) minIdx = j;
        if (minIdx != i) swap(arr[i], arr[minIdx]);
    }
}
```

Сортиране чрез вмъкване (Insertion Sort) — O(n²), стабилен

Идея: поддържаме сортирана лява часть; всяка нова стойност вмъкваме на правилното място.

```
void insertionSort(int* arr, int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

Сравнение:

Алгоритъм	Сложност	Стабилен	Бележка
Bubble Sort	$O(n^2)$	Да	Лесен за имплементация; оптим. с swapped флаг
Selection Sort	$O(n^2)$	Не	По-малко суапове от bubble
Insertion Sort	$O(n^2)$	**Да**	Добър за почти сортирани масиви

---

6. Рекурсия

Рекурсия = функция, която извиква сама себе си.

Задължителни части:

1. **База (дъно)** — граничен случай, без рекурсивно извикване
2. **Стъпка** — свежда по-сложния случай към по-прост

```
f(n) = база      ако n е прост случай
f(n) = ... f(n-1) ...   иначе
```

Видове рекурсия

Вид	Описание	Пример
Пряка	f() извиква директно f()	`factorial`, `fib`
Косвена	f() → g() → f()	Взаимно-рекурсивни функции
Линейна	Едно рекурсивно извикване	`factorial`
Разклонена	Повече от едно рекурсивно извикване	`fib`, DFS

Линейна рекурсия — Факториел

```
long long factorial(int n) {
    if (n == 0) return 1;      // база
    return n * factorial(n-1); // стъпка
}
// factorial(4) → 4 * factorial(3) → 4 * 3 * factorial(2) → ...
```

Опашкова рекурсия (tail recursion) — последното действие е рекурсивното извикване; по-ефективна (компилаторът може да я оптимизира):

```
long long factTail(int n, long long acc = 1) {
    if (n == 0) return acc;
    return factTail(n - 1, acc * n);  // последно действие = рекурсия
}
```

Разклонена рекурсия — Фибоначи

```
int fib(int n) {
    if (n == 0) return 0;  // база
    if (n == 1) return 1;  // база
    return fib(n-1) + fib(n-2);  // две рекурсивни извиквания
}
// Сложност: O(2^n) — много пресмятания се повтарят
```

Оптимизация с memoization:

```
int memo[100] = {-1};  // инициализирай с -1

int fibMemo(int n) {
    if (memo[n] != -1) return memo[n];  // вземи от кеш
    if (n <= 1) return memo[n] = n;
    return memo[n] = fibMemo(n-1) + fibMemo(n-2);
}
// Сложност с memo: O(n)
```

КАК СЕ РЕШАВАТ ЗАДАЧИ

> Тема 12 е директно свързана с **Task 1** на практическия изпит. Форматите:
> **попълни празни места в рекурсия**, **проследи изхода на рекурсивна функция**, **напиши функция с масиви/низове**.

Тип 1: Проследяване на рекурсивна функция

Стратегия: Разписвай извикванията като дърво или стек.

```
int mystery(int n) {
    if (n <= 0) return 0;
    return n + mystery(n - 2);
}
// mystery(5):
//   5 + mystery(3)
//   5 + 3 + mystery(1)
//   5 + 3 + 1 + mystery(-1)
//   5 + 3 + 1 + 0 = 9
```

При разклонена рекурсия — разпиши дървото:

```
fib(4)
├── fib(3)
│   ├── fib(2)
│   │   ├── fib(1) = 1
│   │   └── fib(0) = 0
│   └── fib(1) = 1
└── fib(2)
    ├── fib(1) = 1
    └── fib(0) = 0
fib(4) = 3
```

Тип 2: Попълване на рекурсивна функция

Стратегия: Намери базата и стъпката.

```
// Сума на масив – рекурсивно
int sumArr(int* arr, int n) {
    if (n == ___) return ___;        // n == 0, return 0
    return arr[n-1] + sumArr(arr, ___);  // n-1
}

// Проверка за палиндром – рекурсивно
bool isPalindromeRec(const char* s, int l, int r) {
    if (l >= r) return ___;          // true
    if (s[l] != s[r]) return ___;   // false
    return isPalindromeRec(s, ___, r-1);  // l+1
}
```

Тип 3: Написване на функция с масиви

Обръщане на масив:

```
void reverse(int* arr, int n) {
    for (int i = 0; i < n / 2; i++)
        swap(arr[i], arr[n - 1 - i]);
}
```

Намиране на минимум/максимум:

```
int findMin(int* arr, int n) {
    int min = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] < min) min = arr[i];
    return min;
}
```

Рекурсивен двоично търсене:

```
int bsRec(int* arr, int lo, int hi, int target) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) return bsRec(arr, mid + 1, hi, target);
    return bsRec(arr, lo, mid - 1, target);
}
```

Бърза справка

Концепция	**Ключов факт**
`*ptr`	Дереферириране — стойността
`&x`	Адресът на x
`ptr + n`	Отмества с n × sizeof(T)
`arr[i]` ≡ `*(arr + i)`	Масивът е указател
Масив във функция	Подава се по адрес — промените се

Концепция	**Ключов факт**
	отразяват
`delete[]`	Задължително за `new[]`
`\0`	Терминатор на C-низ
Bubble/Selection Sort	$O(n^2)$, не са стабилни (Selection)
Insertion Sort	$O(n^2)$, **стабилен**
Binary Search	O(log n), нужен е **сортиран** масив
Линейна рекурсия	Едно рекурсивно извикване
Разклонена рекурсия	Повече извиквания, напр. fib — $O(2^n)$
Memoization	Кешира вече изчислени стойности → O(n)
Tail recursion	Рекурсията е последното действие